Serverless Architecture

pay per call, mind the cold start

Watch the 4-min video

In one line: serverless (functions as a service) runs your stateless function when an event arrives, scales it out automatically and bills per request and duration; it fits spiky, event-driven, short tasks and fits poorly with long jobs, steady heavy traffic, strict latency or stateful connections.

1 No servers, no idle cost

What if you never patched a server, never configured autoscaling, and paid nothing when nobody used the app? That is the promise of **FaaS**: AWS Lambda, Azure Functions, Google Cloud Functions (now Cloud Run functions). You upload a function; the platform runs it when an event arrives and runs more copies in parallel as events pile up. Lambda bills per request and per millisecond of duration, scaled by the memory you allocate.

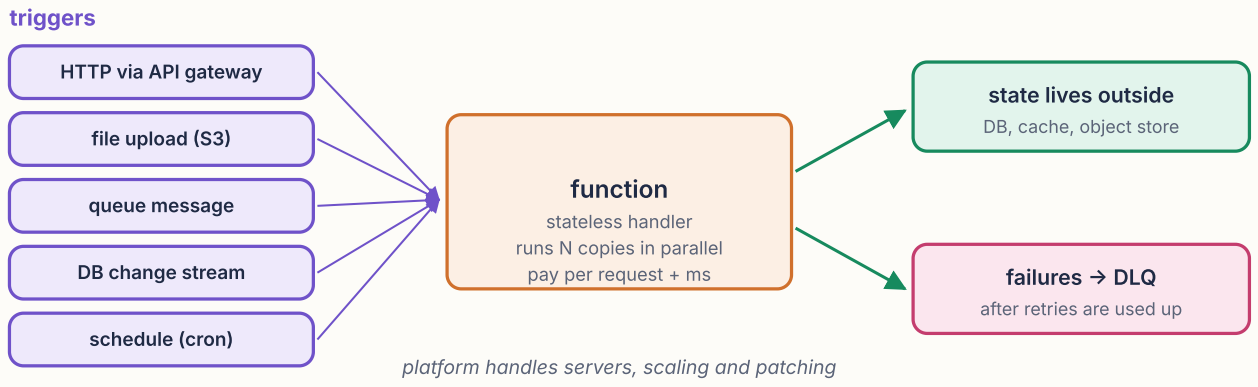


Figure 1. Events trigger stateless functions; state lives in managed services; failed events go to a dead-letter queue.

2 Triggers make it event-driven

An HTTP request through an API gateway, an object uploaded to storage, a message on a queue, a database change stream, a schedule. This makes serverless a natural fit for event-driven glue code and bursty work. Functions can be chained with queues between them to form pipelines, or coordinated by a workflow service such as AWS Step Functions.

3 The cold start

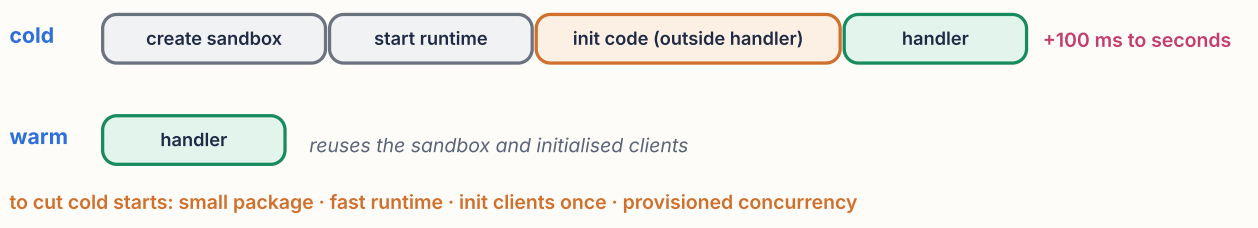


Figure 2. A cold start pays for sandbox creation, runtime start and your init code before the handler runs.

When no warm instance exists, the platform creates a sandbox, starts the runtime and runs your initialisation code. That adds from around 100 ms to several seconds, depending mostly on runtime and package size (large JVM or .NET apps are slowest without tuning). To reduce it: keep packages small, choose a fast-starting runtime, create SDK and database clients **outside the handler** so warm invocations reuse them, and use **provisioned concurrency** (pre-warmed instances, paid for while idle) on latency-sensitive paths.

4 Other limits

Limit	What it means	Mitigation
Stateless	local memory and disk don't survive between calls	keep state in a DB, cache or object store
Max duration	Lambda stops a run at 15 minutes	split work, or use containers or batch jobs
Concurrency	account limit (Lambda default 1,000 per region); fast scale-out can flood a database	reserved concurrency, queue in front, connection proxy (RDS Proxy)
Observability	many short-lived executions	structured logs, correlation IDs, tracing (lesson 36)

WHEN SERVERLESS IS A BAD FIT

- **Long-running jobs** beyond the duration limit.
- **Steady, high-volume traffic:** always-on containers or VMs are usually cheaper per request.
- **Strict low latency** that can't tolerate cold starts (unless you pay for provisioned concurrency).
- **State or persistent connections:** WebSockets held for hours, in-memory caches, local files.

Many teams mix both: containers for the core API, functions for event-driven side jobs.

WORKED EXAMPLE: PHOTO THUMBNAILS

A user uploads a photo to an S3 bucket. The `ObjectCreated` event triggers a Lambda function that downloads the image, creates small, medium and large thumbnails and writes them to a **different** bucket (writing to the same bucket would re-trigger the function in a loop). With no uploads the compute cost is zero; a burst of 10,000 uploads runs many copies in parallel, capped by reserved concurrency. Events that still fail after retries go to a dead-letter queue for inspection.

5 Key terms

FaaS: functions as a service: run code per event

Cold start: delay to create a new function instance

Provisioned concurrency: pre-initialised instances kept warm

Reserved concurrency: cap (and guarantee) for one function

Dead-letter queue: holds events that failed all retries

Trigger: event source that invokes a function

6 Interview questions

Q1. What is serverless / FaaS? BASIC

You deploy functions; the platform runs them on events, scales them automatically and bills per request and execution time, with no servers to manage.

Q2. What is a cold start? BASIC

The extra latency when no warm instance exists and the platform must create a sandbox, start the runtime and run initialisation code.

Q3. When is serverless a bad fit? SCENARIO

Long-running jobs, steady high-volume traffic where always-on servers are cheaper, strict low-latency paths, and workloads needing state or persistent connections.

Q4. How do you handle cold starts? SCENARIO

Keep packages small, initialise clients outside the handler, choose a fast-starting runtime, and use provisioned concurrency on latency-sensitive paths.

Q5. A traffic spike makes your Lambda functions take down the database. Why and how to fix? TRICKY

Functions scale out to hundreds of concurrent copies, each opening connections; cap reserved concurrency, put a queue in front, and pool connections (e.g. RDS Proxy).

Q6. Your S3-triggered function runs for ever and the bill explodes. Likely cause? TRICKY

It writes its output to the same bucket that triggers it, creating a loop; write to another bucket or filter the trigger by prefix.

Next: SD-L-016 · Saga Orchestration backendroadmap.com